

# SigmaQuant: Hardware-Aware Heterogeneous Quantization Method for Edge DNN Inference

Qunyou Liu , Pengbo Yu , Marina Zapater , *Member, IEEE*, and David Atienza , *Fellow, IEEE*

**Abstract**—Deep neural networks (DNNs) are essential for performing advanced tasks on edge or mobile devices, yet their deployment is often hindered by severe resource constraints, including limited memory, energy, and computational power. While uniform quantization provides a straightforward approach to compress model and reduce hardware requirement, it fails to fully leverage the varying robustness across layers, and often lead to accuracy degradation or suboptimal resource usage, particularly at low bitwidths. In contrast, heterogeneous quantization, which allocates different bitwidths to individual layers, can mitigate these drawbacks. Nonetheless, current heterogeneous quantization methods either needs huge brute-force design space search or lacks the adaptability to meet different hardware conditions, such as memory size, energy budget, and latency requirement. Filling these gaps, this work introduces *SigmaQuant*, an adaptive layer-wise heterogeneous quantization framework designed to efficiently balance accuracy and resource usage for varied edge environments without exhaustive search. SigmaQuant allocates layer-wise bitwidths based on weight standard deviation and KL divergence, enabling adaptive quantization under real hardware constraints. This strategy efficiently balances accuracy, memory, and latency without the use of exhaustive search. We validate its practicality through ASIC integration in a shift-add-based accelerator, analyzing power, performance, and area (PPA) trade-offs for the resulting mixed-precision models. Experimentals on CIFAR-100 and ImageNet show that SigmaQuant consistently outperforms both uniform and state-of-the-art heterogeneous quantization. At an equal size model, it achieves up to 2.0% higher accuracy; at an equal accuracy, it reduces memory by up to 40.0%. Hardware evaluation demonstrates up to 22.3% area savings and 20.6% less energy cost compared to the widely used INT8 quantization and implementation, with slight latency overhead and comparable accuracy. These results confirm the effectiveness of SigmaQuant for edge AI deployment.

**Index Terms**—Hardware accelerator, neural network, algorithm, KL divergence.

Received 21 June 2025; revised 26 September 2025 and 21 December 2025; accepted 6 February 2026. Date of publication 20 February 2026; date of current version 29 May 2026. This work was supported in part by Swiss State Secretariat for Education, Research, and Innovation (SERI) through the SwissChips research project, and in part by Intel as part of the Intel Center for Heterogeneous Integrated Platforms (HIP). The review of this article was arranged by Associate Editor Chenchen Liu. (*Corresponding author: Qunyou Liu.*)

Qunyou Liu, Pengbo Yu, and David Atienza are with the Embedded Systems Laboratory (ESL), EPFL, 1015 Lausanne, Switzerland (e-mail: qunyou.liu@epfl.ch; pengbo.yu@epfl.ch; david.atienza@epfl.ch).

Marina Zapater is with the University of Applied Sciences and Arts Western Switzerland (HES-SO), 1401 Yverdon-les-Bains, Switzerland (e-mail: marina.zapater@heig-vd.ch).

Digital Object Identifier 10.1109/TCASAI.2026.3666506

## I. INTRODUCTION

DEEP Neural Networks (DNNs) have become the backbone of many edge computing applications, ranging from image processing on devices [1] to speech recognition [2] and beyond. However, executing these models with strict resource budgets remains a formidable challenge. In particular, edge devices often have extremely limited memory size [3], [4], processing capabilities, and stringent energy supplies, which can make large and high-precision DNNs prohibitively expensive or even infeasible to deploy [5], [6].

Quantization has emerged as one of the most effective strategies to tackle this problem, as it converts 32-bit floating point (FP) weights and activations into lower-precision representations (e.g., 8-bit or even 4-bit integers). This compression reduces the storage footprint and computational overhead of the model [7], [8]. However, *uniform* quantization, where all layers share the same bitwidth, often fails to achieve an optimal trade-off between accuracy and efficiency. The reason is that each DNN layer exhibits distinct statistical properties and different robustness to quantization noise [9], [10]. Therefore, forcing a single global precision can over-allocate bits to some layers and under-allocate them to others.

To overcome these limitations, *heterogeneous* quantization has recently gained popularity. This approach tailors the bitwidth per layer based on the quantization sensitivity of each layer, resulting in more compact models with minimal accuracy loss. However, existing methods typically rely on layer-wise sensitivity heuristics or resource-intensive search algorithms (e.g., reinforcement learning) [11], [12]. In addition, edge devices exhibit various hardware configurations and resource constraints. For example, IoT sensors might have only a few megabytes of available main memory, whereas mobile phones can offer significantly more but require higher accuracy for applications like real-time image recognition or speech processing. Consequently, a static quantization scheme optimized for one scenario often fails to generalize to other hardware deployments. Formally, the quantization process should ideally satisfy *boundary conditions* on memory and accuracy constraints:

$$\text{Memory Usage} \leq \text{Memory Constraint},$$

$$\text{Accuracy} \geq \text{Accuracy Constraint}.$$

Current heterogeneous quantization methods fail to satisfy this adaptive requirement, as they typically provide fixed solutions that cannot accommodate the diverse and dynamic demands of edge scenarios. Beyond algorithmic efficiency,

heterogeneous quantization must also deliver tangible hardware benefits, particularly when deployed on edge AI accelerators. These accelerators increasingly rely on reduced-precision arithmetic to efficiently minimize area, power, and latency. The widely adopted shift-add scheme for multiply-accumulate (MAC) operations constitutes a representative example [13], [14], [15], [16]. In such cases, lower bitwidths significantly accelerate MAC computation, as fewer bits directly translate into less processing cycles and lower energy consumption. However, simply applying the lowest precision everywhere is ineffective, since some layers are highly sensitive to quantization noise. Over-quantizing these layers can lead to severe accuracy loss, undermining any hardware gains [17], [18]. This motivates a hardware-aware heterogeneous quantization strategy that carefully allocates precision across the network—assigning low bitwidths where feasible and preserving higher precision where needed. Such an approach ensures compliance with hardware constraints (e.g., memory size, energy budgets, real-time latency) and maximizes the hardware efficiency.

For this purpose, we propose SigmaQuant, a hardware-aware heterogeneous quantization framework that adaptively allocates per-layer bitwidth under user-specified memory and accuracy targets, while fully leveraging hardware efficiency simultaneously. SigmaQuant has two search phases: (i) Phase-1 provides a stable, constraint-aware initialization. All layers are grouped by simple distribution statistics so that the model quickly moves into a region that satisfies at least one boundary (memory size or accuracy). (ii) Phase-2 then performs fine-grained adjustments on a few layers each time to meet both targets while avoiding unnecessary changes in most layers. The proposed two-phase scheme reduces the search overhead and effectively satisfies the requirements imposed by tight resource budgets. We employ *SigmaQuant* on CIFAR-100 and ImageNet datasets and on a set of DNNs (e.g., ResNet and MobileNet) and compare performance with respect to a uniform quantization scheme and several state-of-the-art heterogeneous quantization techniques. Furthermore, we evaluate *SigmaQuant* on a general shift-add-based MAC unit widely used in modern edge accelerators, demonstrating its effectiveness in reducing latency and energy. We also provide a detailed trade-off analysis that highlights the co-optimization between software quantization strategies and hardware design, showing how SigmaQuant impacts performance, power, and area (PPA) metrics. This underscores the importance of proposing quantization methods that simultaneously optimize algorithmic performance while adhering to hardware constraints for efficient deployment. More specifically, the **key contributions** of our work are the following:

- We introduce a **distribution-based approach** to study heterogeneous quantization, fully utilizing the weight of each layer, guided by standard deviation and KL divergence.
- We develop a **two-phase quantization method** that combines a cluster-based assignment with iterative divergence-driven refinement, striking an effective accuracy–efficiency balance in a reasonable design space search. Our method adapts the layer bitwidth assignments according to the user-defined *boundary conditions* on

accuracy and model size, ensuring minimal quantization loss while aggressively compressing the network.

- We **validate our method** on CIFAR-100 [19] and ImageNet [20] with popular DNN architectures (mainly ResNet and MobileNet families). Compared to state-of-the-art heterogeneous schemes, our approach (1) reduces memory usage by up to **17.7%** while maintaining or exceeding accuracy and (2) reaches over 2% in Top-1 accuracy under similar memory budgets. Compared to uniform quantization, *SigmaQuant* can reach the same accuracy with only 60% memory budget, and achieve 4% more accuracy with the same model size.
- We evaluate **SigmaQuant** using a generic hardware implementation based on the shift-add scheme, which is widely used in edge scenarios. The results show that our method provides plenty of quantization choices between latency/energy and accuracy. It outperforms uniform quantization schemes such as A8W4 in all metrics. Compared to a well-performing INT8 quantization strategy and hardware, **SigmaQuant** can further reduce area by 22.3% and energy consumption by up to 20.6% with slight latency overhead and comparable accuracy.

The remainder of this paper is organized as follows. Section II describes the most relevant heterogeneous quantization research in the state of the art. Section III explains the mathematical explanation for the algorithm and hardware background of suitable accelerator used in performance validation. Section IV details our distribution-fitting approach and two-phase algorithm. Section V describes our experimental setup, followed by results and analysis in Section VI. We conclude and discuss future directions in Section VII.

## II. RELATED WORK

Deep neural network quantization has seen extensive research to balance accuracy and efficiency for deployment on resource-constrained hardware. A common approach is uniform quantization, where all weights and activations are quantized to a fixed bitwidth using a linear mapping scale. Uniform low-precision (e.g., 8-bit integer) models can significantly reduce memory usage and speed up inference with only minor accuracy loss in many cases [11]. However, pushing uniform quantization to ultra-low bitwidths (4-bit or below) often incurs severe accuracy degradation [11]. Techniques such as quantization-aware training and careful calibration improve the situation [11], but ultimately a one-size-fits-all bitwidth may not suit all layers. Some layers are more sensitive to quantization errors than others (as described in Section III-A), so using the same precision budget for every layer can be suboptimal (as shown in Section VI-C). This rigidity motivates the use of adaptive quantization strategies.

Non-uniform quantization techniques allocate quantization levels unevenly, guided by the sensitivity of layers. Rather than spacing quantization levels uniformly, these methods concentrate precision where it matters the most (e.g., large-magnitude values). For instance, clustering or entropy-based schemes learn value clusters that maximize information content [21],

effectively compressing many small-magnitude weights into fewer levels while reserving more levels for important outliers [21]. Zhu et al. propose an entropy-aware layerwise quantization where the bitwidth of each layer is chosen based on the weight distribution entropy, allowing more complex layers to use higher precision (and vice versa) [22]. Baskin et al. [12] introduce a non-uniform  $k$ -quantile approach (UNIQ) that adapts the quantizer to the parameter distribution by injecting noise during training to emulate a quantile-based quantization. These non-uniform quantizers can better preserve accuracy under aggressive compression, especially in the low-bit regime [12], but their irregular quantization levels offer only a limited set of solutions and may not reliably meet memory constraints.

Recent approaches formulate mixed-precision selection as an optimization problem under hardware constraints. Hardware-Aware Quantization frameworks like HAQ [23] employ reinforcement learning to search for the Pareto-optimal bit allocation across layers given a target memory or latency budget, automating mixed-precision selection. Its runtime depends on the training budget and on the details of the implementation. Likewise, differentiable Neural Architecture Search [24] techniques have been applied to quantization, learning per-layer precisions via gradient-based optimization [25], but this approach entails training a large super-network (for bitwidth choices) – incurring considerable computational overhead – and can be sensitive to initialization and hyper-parameters. Second-order sensitivity analysis is also well established in statistics and machine learning [26], [27]. An alternative line of work uses second-order sensitivity (HAWQ – Hessian-Aware Quantization). HAWQ leverages the Hessian spectrum of each layer to gauge how quantization will impact the loss [27]. Heterogeneous DNNs have demonstrated that judiciously allocating higher bitwidth to error-sensitive layers and lower precision to tolerant layers can reduce model size and inference cost with only a small accuracy drop.

There are also approaches for dynamic or adaptive precision at runtime. Instead of fixing a mixed-precision configuration post-training, a single model is trained to support multiple bitwidth settings. Jin et al. [28] propose AdaBits, which enables a network to switch between bitwidths on the fly (e.g., 2-bit to 8-bit) without retraining. By joint training with adaptive bitwidths and using techniques like switchable clipping levels for activations, they obtain a model that can cater to different hardware capabilities or energy budgets. This adds a new dimension of flexibility: one deployed model can adapt to various scenarios. The trade-off is a more involved training procedure (to ensure the network performs well at all supported precisions) and a slight runtime overhead to manage precision switching.

Existing network quantization methods, including uniform and mixed-precision approaches, often do not explicitly minimize the divergence between the pre- and post-quantization distributions of weights or activations. Instead, they use indirect criteria (e.g. minimizing quantization noise variance or heuristics like entropy/KL to set ranges), which do not guarantee that the quantized values preserve the original distribution [29]. This oversight can lead to a mismatch in distributions that degrades

accuracy, as recently noted by several studies [29] [30]. Moreover, most quantization frameworks produce a fixed precision configuration for a model. Many mixed-precision techniques explore only a handful of bitwidth assignments and then adopt a static scheme, which lacks flexibility across different hardware settings [31]. This nature means the model cannot easily adapt its bitwidth per layer to meet the constraints or capabilities of new devices. In summary, earlier quantization methods did not explicitly align distributions and often yielded fixed precision setups.

While heterogeneous quantization effectively tailors precision to each layer's tolerance, deployment on real hardware demands further consideration. Edge AI accelerators, whether ASIC, DSIP, DSP, FPGA, etc., operate under strict area, power, and latency constraints. Reduced-precision arithmetic directly affects all these metrics, as lowering bitwidth reduces MAC complexity, diminishes the memory footprint and data movement, and thereby decreases power consumption. For example, for the shift-add scheme widely used in many accelerators, each additional bit in a multiplier operand contributes an extra cycle and corresponding power cost. Thus, lowering the weight bitwidth yields an approximately proportional reduction in compute latency and energy consumption per operation. However, naively pushing all layers to the lowest precision can devastate accuracy, since sensitive layers quickly lose accuracy under over-quantization. Existing quantization techniques largely neglect this co-optimization. Many prior methods focus on accuracy and model size (often via heuristic or search-based bit allocations) without explicitly modeling hardware costs. These approaches yield fixed precision assignments that may satisfy a bit-budget or target accuracy but run afoul of practical limits like on-chip memory capacity, power budgets, or throughput constraints. In other words, a model might be well-compressed in theory but still too slow or energy-hungry for the target hardware. This gap in prior work motivates a hardware-aware solution that jointly addresses algorithmic and hardware objectives. Beyond early mixed-precision schemes such as HAQ and HAWQ, recent work has progressed along several complementary directions that explicitly consider hardware constraints and/or more expressive search formulations. On the hardware–software co-design side, Edge-MPQ [32] integrates versatile mixed-precision inference units into a RISC-V pipeline and couples them with a hardware-aware layerwise MPQ search, targeting latency and energy on a specific edge platform. On the training/sensitivity side, Huang et al. [33] propose HMQAT, a Hessian-based mixed-precision QAT framework that drives bit configuration via a Pareto-frontier search and further improves stability through transition-aware fine-tuning of quantization scales. From the perspective of joint compression under an accelerator objective, Balaskas et al. [34] explore a combined pruning–quantization design space with a reinforcement-learning agent guided by an accelerator-level energy model. Finally, Deng et al. [35] present CLADO, which explicitly models cross-layer dependencies of quantization errors and formulates bitwidth assignment as an integer quadratic program to obtain strong accuracy–compression trade-offs.



Motivated by these advances and their practical trade-offs, our proposed approach, *SigmaQuant*, adopts a lightweight yet hardware-aware distribution-fitting perspective for heterogeneous quantization. Specifically, *SigmaQuant* uses layer-wise standard deviation and the KL divergence between the floating-point and quantized distributions to guide bitwidth assignment in a two-phase procedure that enforces user-defined accuracy and resource constraints (model size or BOPs) without exhaustive search, while remaining compatible with integer-only shift-add accelerators.

### III. BACKGROUND

In this section, we first review the conventional (uniform) quantization scheme and then introduce our perspective of approximating each layer's weight distribution by a discrete (quantized) distribution. This *distribution-fitting* viewpoint motivates why different layers may require distinct bitwidths. We also highlight the increasing importance of hardware-constrained design, wherein bitwidth choices must align with metrics such as model size to meet resource and latency targets on edge devices, and we provide some background on the specific hardware design that we use to showcase the results of our work.

#### A. Model Quantization and Distribution

1) *Uniform Quantization Basics.*: Let  $\mathbf{w} \in \mathbb{R}^n$  denote the set of weights in a given DNN layer. In a uniform quantization scheme with integer bitwidth  $b$ , each weight  $w_i$  is replaced by a quantized value

$$\tilde{w}_i = \text{clip}\left(\text{round}\left(\frac{w_i}{\Delta}\right), -Q, Q\right) \times \Delta,$$

where  $Q = 2^{(b-1)} - 1$  (for signed quantization) and  $\Delta$  is the quantization step size (or scale). Typically,  $\Delta$  is chosen in one of two ways:

- *Max-based scaling*, where  $\Delta = \max_i |w_i|/Q$ ,
- *Statistical scaling*, where  $\Delta = k\sigma$ , with  $\sigma$  denoting the standard deviation of  $\mathbf{w}$  (and  $k$  a chosen constant, e.g., 2 or 3).

All weights are thus mapped to one of  $2^b$  uniformly spaced levels in  $[-Q\Delta, Q\Delta]$ . While straightforward, a global uniform bitwidth can be suboptimal because different layers exhibit widely varying sensitivities to quantization noise [11], [12], [27], [36]. This observation motivates *heterogeneous quantization*, wherein each layer (or channel) is allowed its own bitwidth  $b_\ell$ . Such layer-wise customization yields better accuracy–size trade-offs: for example, early convolutional layers often have smaller weight variance and can safely be quantized to 4 bits, whereas layers with broader weight distributions may require 8 bits to retain accuracy. The challenge, then, is to determine how to assign precision across layers in a principled yet efficient way.

2) *Quantization Sensitivity and Standard Deviation.*: We conducted preliminary experiments to identify a simple indicator of layer-wise quantization sensitivity. Using ImageNet [20] as dataset and Alexnet [37] and ResNet18 [38] as example

TABLE I  
OBSERVATIONS ON A BASELINE (BOP-BASED) HEURISTIC VS. FINAL BITWIDTH AND WEIGHT DISTRIBUTION FOR ALEXNET (IMAGENET). HIGHER STANDARD DEVIATION  $\sigma$  CORRELATES WITH THE NEED FOR HIGHER BITWIDTH TO KEEP THE KL DIVERGENCE  $D_{KL}$  LOW

| Layer           | Init Bits | Final Bits | $\sigma$ | $D_{KL}$ |
|-----------------|-----------|------------|----------|----------|
| AlexNet – Conv1 | 8         | 6          | 0.115672 | 0.022229 |
| AlexNet – Conv2 | 8         | 6          | 0.046543 | 0.000626 |
| AlexNet – Conv3 | 8         | 4          | 0.034646 | 0.000477 |
| AlexNet – Conv4 | 8         | 4          | 0.027320 | 0.000409 |
| AlexNet – Conv5 | 8         | 4          | 0.026128 | 0.000619 |
| AlexNet – FC1   | 8         | 2          | 0.009245 | 0.000097 |
| AlexNet – FC2   | 8         | 2          | 0.011537 | 0.000104 |
| AlexNet – FC3   | 8         | 4          | 0.018524 | 0.000136 |

models, we applied progressively lower bitwidths per layer to reach minimum Bits Operations (BOPs) [12] under a fixed overall precision budget using brute-force search. A remarkably strong correlation emerged between each layer's acceptable bitwidth and the standard deviation  $\sigma$  (*Sigma*) of its weights: layers with relatively low  $\sigma$  tolerated very low precision (4-bit or even 2-bit) with minimal accuracy impact, whereas layers with high  $\sigma$  either needed to remain at higher precision (e.g., 8-bit) or suffered severe accuracy drops when forced to fewer bits. Table I illustrates this trend for AlexNet running on ImageNet, where an initial heuristic assignment (uniform 8-bit for all layers) is compared against the final adjusted bitwidths. Layers with larger  $\sigma$  (e.g., Conv1) ended up requiring higher  $b_\ell$  to avoid excessive quantization error, as reflected by the larger KL divergence  $D_{KL}$  between the original and quantized weight distributions for those layers. In contrast, layers with small  $\sigma$  (e.g., the fully-connected layers) were quantized to 2 bits with negligible  $D_{KL}$  penalty. ResNet18 shows the same trend.

These empirical findings confirm that the weight standard deviation  $\sigma_\ell$  is a strong first-order indicator of layer  $\ell$ 's quantization sensitivity. Intuitively,  $\sigma_\ell$  gauges the “width” of the weight distribution: a large spread means more information can be lost when mapping to a small set of levels, whereas a tightly concentrated distribution can be compressed more aggressively. This insight aligns with prior observations that layer-wise weight distributions influence quantization outcomes. For instance, the quantization whitepaper in [9] recommends setting the quantizer range based on multiples of the weight standard deviation, and Zhu et al. propose an entropy-based bit allocation per layer to account for distribution complexity [22]. More sophisticated frameworks have also quantified layer sensitivity via reinforcement learning or second-order analysis (Hessian) [11], reinforcing the notion that certain layers are inherently more error-prone under quantization. Our results show that a simple statistic  $\sigma_\ell$  can serve as an effective proxy for this sensitivity, providing a convenient starting point for assigning heterogeneous bitwidths. In practice, this means we can sort or cluster layers by  $\sigma_\ell$  and assign lower precisions to those with small  $\sigma_\ell$  while reserving higher bits for layers with large  $\sigma_\ell$ , before applying any fine-grained adjustments.

3) *Distribution-Fitting Perspective.*: While  $\sigma$  provides a useful heuristic, we also adopt a theoretical perspective to quantify quantization loss more rigorously. Consider the

empirical distribution of the weights in layer  $\ell$ , denoted  $p_\ell(w)$ . We can write

$$p_\ell(w) = \frac{1}{|W_\ell|} \sum_{w_i \in W_\ell} \delta(w - w_i),$$

where  $W_\ell = \{w_i\}$  is the set of weights and  $\delta(\cdot)$  is the Dirac delta (this can be thought of as the normalized weight histogram for layer  $\ell$ ). After quantizing layer  $\ell$  to  $b_\ell$  bits, the set of quantized weights  $\{\tilde{w}_i\}$  induces a discrete distribution  $\tilde{p}_\ell(w)$  supported only on the finite quantization levels. Rather than viewing quantization merely as per-weight rounding error, we regard it as an *information-theoretic approximation* of  $p_\ell$  by  $\tilde{p}_\ell$ . A natural measure of the mismatch between these two distributions is the Kullback–Leibler (KL) divergence:

$$D_{KL}(p_\ell \| \tilde{p}_\ell) = \sum_w p_\ell(w) \log \frac{p_\ell(w)}{\tilde{p}_\ell(w)}. \quad (1)$$

When the quantization step size (represented by  $b_l$ ) is too small compared to the range of values in  $W_l$ , the quantized representation  $\tilde{p}_l$  will not capture the differences between many distinct values, leading to a loss of precision. Conversely, a sufficiently high  $b_\ell$  yields  $\tilde{p}_\ell$  that closely approximates  $p_\ell$ , resulting in a small KL divergence. In essence, choosing an appropriate bitwidth  $b_\ell$  amounts to balancing model size (or efficiency) against an acceptable level of distribution distortion. This distribution-fitting view provides a theoretical foundation for quantization: one seeks to minimize information loss by ensuring  $\tilde{p}_\ell$  is a good fit to  $p_\ell$ .

Importantly, this perspective helps explain the earlier empirical trend with  $\sigma_\ell$ . A layer with a large  $\sigma_\ell$  typically has a broad  $p_\ell(w)$ ; quantizing it aggressively (small  $b_\ell$ ) forces a poor approximation  $\tilde{p}_\ell(w)$ , yielding a high  $D_{KL}$  and, ultimately, higher accuracy degradation. In contrast, a layer with small  $\sigma_\ell$  (narrow distribution) can be quantized to low precision with only a minor increase in  $D_{KL}$ . In practice, we leverage  $D_{KL}$  as a quantification of quantization “distortion” to complement the  $\sigma_\ell$  heuristic. Recent studies similarly note that preserving the original distribution (or minimizing distribution shift) is critical for maintaining accuracy [39]. Thus, by using  $\sigma_\ell$  as a first-order guide and  $D_{KL}(p_\ell \| \tilde{p}_\ell)$  as a refinement criterion, we ground our heterogeneous quantization strategy in both empirical intuition and theoretical approximation. Subsequent sections will reference these metrics as the basis for our two-phase bitwidth assignment algorithm, without needing to repeat the justification for why they are chosen.

### B. Exploiting SigmaQuant in Efficient Shift-Add-Based Accelerators for Edge Devices

Modern edge and mobile devices are constrained by limited memory, computational power, and energy resources, which significantly hampers DNN inference on these platforms. To increase the speed-up and efficiency of inference in resource-constrained platforms, models are quantized and executed on dedicated or domain-specific hardware accelerators.

The core computation behind DNNs are matrix multiplications (GEMMs) and convolutions, both of which are fundamentally composed by MAC operations, as presented in Fig. 1(a).

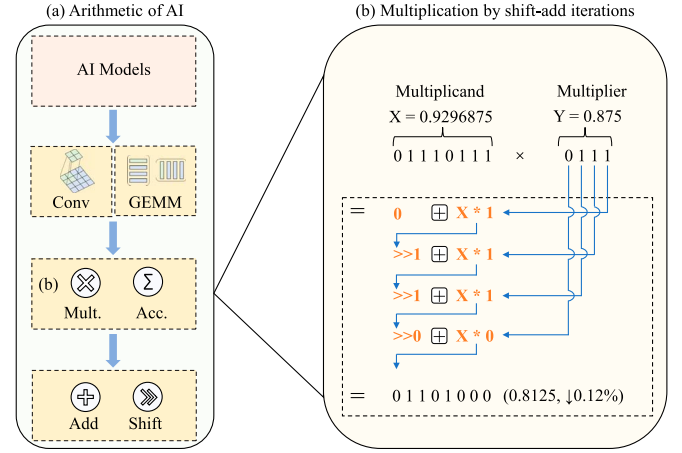


Fig. 1. (a) The operation hierarchy of AI models from top to down, and (b) the widely used shift-add-based multiplier for edge accelerators that target at high energy efficiency.

Consequently, enhancing the efficiency of multiplication and accumulation is crucial. In this context, shift-add-based multipliers [13], [14], [15], [16] are widely employed in hardware accelerators, as they can reduce area consumption, lower power costs, and relax critical timing constraints. Fig. 1(b) illustrates an example of an 8-bit  $\times$  4-bit multiplication, yielding an 8-bit result via iterative shift-add operations. The input operands are in Q1.X format, that 1-bit for integer (i.e., 0 or -1) and others for fraction, ranging in (-1,1). The process begins at the least significant bit (LSB) of the multiplier, with each bit processed sequentially through an addition followed by a right-shift, and concludes at the most significant bit (MSB). The multiplication result is truncated to 8 bits, that is the same size of the multiplicand, since truncation loss is often minimal (only -0.12% in this example).

However, while the shift-and-add scheme offers significant area and energy efficiency, it is constrained by increased latency, as a naive implementation requires  $n$ -cycle iterations for an  $n$ -bit multiplier operand. Consequently, reducing the size of the multiplier operand can substantially improve latency performance and further enhance energy efficiency. Although techniques such as executing multiple shift operations for trailing zeros (e.g., “1000”) within a single cycle [16], [40] or employing Canonical Signed Digit (CSD, e.g., 0111 to 100-) coding [16] for the multiplier have been proposed to mitigate this issue, the overall latency remains strongly correlated with the size of the multiplier operand. For DNNs, the multiplier operand typically corresponds to the weight. Therefore, using lower-bitwidth weights can eliminate latency concerns while simultaneously yielding additional reductions in energy consumption.

Therefore, in this work, we target a generic shift-add based arithmetic scheme to assess the benefits of our proposed quantization method relative to uniform quantization baselines, with the evaluation reflecting arithmetic efficiency rather than being bound to any particular hardware platform. More detailed discussion about model size, latency, energy consumption are provided in Section VI.

## IV. METHODOLOGY

### A. Algorithm Overview

Based on our preliminary experiments and observations, *SigmaQuant* proposes a two-phase procedure to determine per-layer quantization bitwidths, focusing first on a coarse clustering assignment and then on fine-grained adjustments. The goal of this section is to explain the structure and logic of this algorithm (summarized in Algorithm 1) without delving into low-level implementation or mathematical derivations. We take advantage of two key metrics – **weight standard deviation** (*Sigma*) and **Kullback–Leibler** (*KL*) divergence – to guide decisions (see Section III-A for the mathematical foundation and preliminary experiments results of these metrics). In essence, Phase 1 provides an initial clustering-based bitwidth assignment, and Phase 2 provides an iterative, divergence-based refinement. Throughout the procedure, the algorithm tracks the accuracy and size of the model relative to these targets (with some buffer tolerance,  $\Delta A$  for accuracy and  $\Delta M$  for size) and uses two key metrics to guide decisions: weight standard deviation (denoted  $\sigma$ , a measure of the spread of the distribution of a layer) and Kullback-Leibler (KL) divergence between the original and quantized weight distributions. Algorithm 1 outlines the overall procedure at a high level, summarizing the two phases described below.

- 1) **Phase 1 - Initial Phase (Cluster-Based Bitwidth Assignment):** The algorithm clusters the layers based on standard deviation  $\sigma$  using an adaptive  $k$ -means method (with penalty parameter  $\lambda$ ). Each cluster is mapped to a target bitwidth (e.g., from the set  $\{2, 4, 6, 8\}$ ). After quantization and calibration, the model's accuracy and size are compared against the desired targets (with buffers  $\Delta A$  and  $\Delta M$ ). If neither metric is acceptable, the clustering is refined (by increasing  $\lambda$ ) and the process repeats until at least one metric meets its boundary.
- 2) **Phase 2 - Refinement Phase (Iterative Improvement):** The algorithm fine-tunes the bitwidths of individual layers. A sensitivity score—combining  $\sigma$  and the KL divergence between the full-precision and quantized weight distributions—is computed for each layer. Layers with high sensitivity (i.e., those most critical to accuracy) are adjusted by increasing their bitwidth, while layers with low sensitivity may have their bitwidth reduced to save memory. This iterative refinement continues until both accuracy and model size meet the targets, thereby placing the model in the Target Zone.

Fig. 2 illustrates this two-phase procedure on a plot of model accuracy (y-axis) versus model size (x-axis). In the diagram, color-coded regions indicate the algorithm's decision zones. We preserve those visual cues here to explain how the algorithm navigates the accuracy–model size trade-off:

- **Bit Increase Zone (Phase 1 light green region):** The model's accuracy is too low, yet its current size is comfortably below the target budget. In this scenario, accuracy is the bottleneck. The algorithm responds by increasing bitwidths for certain layers (typically those most sensitive to quantization error) to regain accuracy

### Algorithm 1 Two-Phase Heterogeneous Quantization

---

**Require:** Float model  $\mathcal{M}$  with  $N$  layers; valid bit-set  $\mathcal{B} = \{2, 4, 6, 8\}$ ; targets  $A_t, M_t$ ; tolerances  $\Delta A, \Delta M$ ; max iters  $I_{\max}$

**Ensure:** Quantised model  $\mathcal{M}_q$

```

1:  $\mathcal{M}_q \leftarrow \text{Initialize8Bit}(\mathcal{M})$  ▷ start with uniform 8-bit
2:  $A \leftarrow \text{Evaluate}(\mathcal{M}_q)$ 
3:  $M \leftarrow \text{ModelSize}(\mathcal{M}_q)$ 
— Phase 1: adaptive clustering —
4:  $\lambda \leftarrow 0.1$ ;  $i \leftarrow 0$ 
5: while  $(A < A_t - \Delta A) \wedge (M > M_t + \Delta M) \wedge (i < I_{\max})$  do
6:    $i \leftarrow i + 1$ 
7:    $feat \leftarrow \text{StdDevFeatures}(\mathcal{M}_q)$ 
8:    $\mathcal{C} \leftarrow \text{AdaptiveKMeans}(feat, 4, \lambda)$ 
9:    $\text{AssignBitwidths}(\mathcal{C}, \mathcal{M}_q, \mathcal{B})$ 
10:   $\text{Calibrate}(\mathcal{M}_q)$ ;  $\text{QAT}(\mathcal{M}_q)$ 
11:   $A \leftarrow \text{Evaluate}(\mathcal{M}_q)$ ;  $M \leftarrow \text{ModelSize}(\mathcal{M}_q)$ 
12:  if  $(A \geq A_t - \Delta A) \vee (M \leq M_t + \Delta M)$  then
13:    break ▷ one metric is inside the buffer
14:  else
15:     $\lambda \leftarrow \lambda + 0.1$ 
16:  end if
17: end while
18: if  $(A < A_t - \Delta A) \wedge (M > M_t + \Delta M)$  then
19:   return  $\mathcal{M}_q$  ▷ give up – infeasible
20: end if
— Phase 2: iterative refinement —
21:  $j \leftarrow 0$ 
22: while  $j < I_{\max}$  do
23:    $j \leftarrow j + 1$ 
24:    $\text{ImproveBitwidths}(\mathcal{M}_q)$ 
25:    $\text{Calibrate}(\mathcal{M}_q)$ ;  $\text{QAT}(\mathcal{M}_q)$ 
26:    $A \leftarrow \text{Evaluate}(\mathcal{M}_q)$ ;  $M \leftarrow \text{ModelSize}(\mathcal{M}_q)$ 
27:   if  $(A \geq A_t) \wedge (M \leq M_t)$  then
28:     break
29:   end if
30: end while
31: return  $\mathcal{M}_q$ 

```

---

- **Bit Decrease Zone (Phase 1 light red region):** The model's accuracy is acceptable (at or above the required threshold), but the model size exceeds the target memory budget. Here, efficiency is the problem. The algorithm decreases bitwidths for some less-sensitive layers to compress the model and reduce its size
- **Transition Zone (middle gray-green region):** Continue the previous cluster trend until reaching the next zone.
- **Abandon Zone (gray region):** Neither accuracy nor model size is anywhere near acceptable levels. In this rare case, the procedure may terminate early to avoid wasting resources on a hopeless quantization scenario (i.e., the chosen constraints might be unattainable simultaneously). Terminate process.
- **Iteration Zone (Phase 2 region):** This zone represents the intermediate state where one of the two criteria (accuracy or size) has met its target, but the other one has not yet met it. The model stays in this zone during the Phase 2 refinement process. The algorithm will perform iterative bitwidth tweaks – increasing or decreasing precision for selective layers – to improve the remaining metric while maintaining the already satisfied metric, effectively nudging the model toward the Target Zone.



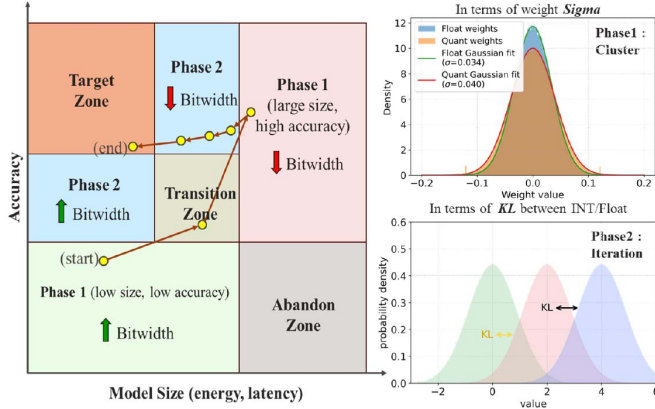


Fig. 2. Overview of our proposed distribution-fitting quantization method. We start from user-defined boundary conditions (target model size and accuracy) and adapt bitwidths in two phases: initial clustering by standard deviation followed by iterative KL-based refinement.

- **Target Zone (light orange region):** Both accuracy and model size meet their target thresholds (within the allowed buffers). When the model lands in this zone, it means it satisfies the accuracy requirement and does not exceed the size budget.

### B. Phase 1: Adaptive Clustering

In the first phase, we assign initial bitwidths to each layer by clustering their standard deviations into  $K = 4$  groups, targeting for the bitwidths of 2-, 4-, 6-, 8-bit. For the initial assignment, we use the conventional k-mean. Then we check the location of the initial point in the Fig. 2, before we decide to increase the bitwidth or decrease the bitwidth. Next, unlike standard  $k$ -means, we introduce a adaptive term that discourages certain cluster from becoming too large, thus promoting a more uniform distribution of layers across the available bitwidths. Concretely, let  $X = \{x_1, x_2, \dots, x_N\}$  be the set of  $N$  layers we wish to cluster, where  $x_i$  is a one-dimensional feature (e.g. standard deviation  $\sigma_i$ ). We seek a partition

$$\mathcal{C} = \{C_1, C_2, \dots, C_K\},$$

of  $X$  into  $K$  clusters. We define the centroid of  $C_j$  as

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x.$$

The *adaptive k-means* objective reads

$$\min_{\mathcal{C}, \mu_j} \sum_{j=1}^K \left[ \sum_{x \in C_j} \|x - \mu_j\|^2 + \lambda \left( |C_j| - \frac{N}{K} \right)^2 \right], \quad (2)$$

where  $\lambda$  controls how strongly we penalize deviations from the “ideal” cluster size  $\frac{N}{K}$ . For each iteration:

- We compute the distances from each layer  $x_i$  to the centroid of each cluster, adjusted by the cluster-size penalty term  $\lambda$ .

- We reassign  $x_i$  to the cluster that minimizes its total cost.
- We update the centroids  $\mu_j$  after all points are reassigned.

We initialize  $\lambda$  to a small value (e.g., 0.1) and gradually increase it (in increments, such as 0.1) whenever the resulting bitwidth assignment fails to meet either of the global buffer conditions ( $\Delta A$  and  $\Delta M$  in 1). Each time a new clustering  $\mathcal{C}$  is obtained, we map each cluster  $C_j$  to a target bitwidth  $b_j \in \{2, 4, 6, 8\}$ . We then calibrate and perform a short quantization-aware training (QAT) cycle. If neither accuracy nor model size usage falls within the acceptable range, we increment  $\lambda$  and repeat. If, after a fixed maximum number of increments, both metrics remain unacceptably far from their targets, the algorithm halts (“abandoning” the attempt and announcing the failure of quantization). We perform a quick calibration of the quantized model before QAT by using a subset of training data to adjust batchnorm statistics and quantization scales (ensuring stable QAT initialization).

### C. Phase 2: Iterative KL-Based Refinement

Once Phase 1 ensures that at least *one* of the target constraints (accuracy or model sizes) is satisfied, we proceed to a finer-grained iterative improvement of the per-layer bitwidths. Here, we focus on adjusting bitwidths to fix whichever metric remains suboptimal. We define a sensitivity measure for each layer  $\ell$  that combines its standard deviation and a normalized Kullback–Leibler (KL) divergence between the float and quantized weight distributions. We use the KL divergence as described in Equation 1. We use a normalized version  $\hat{D}_{KL}$  to bound it between 0 and 1, e.g. by dividing by  $D_{KL}(p_\ell \| p_{\text{int8}})$  for an 8-bit baseline distribution  $p_{\text{int8}}$ . In each iteration of Phase 2 we perform the following actions:

- 1) **Measure Sensitivity:** For layer  $\ell$ , we define a sensitivity score

$$\hat{D}_{KL}(p_\ell \| \tilde{p}_\ell),$$

Layers with a higher  $\hat{D}_{KL}$  are more critical to accuracy, so if accuracy is below the target, we *increase* their bitwidth first. Conversely, if model size usage is too high, we *decrease* bitwidth on layers with a lower sensitivity score.

- 2) **Apply Changes & Calibrate:** We pick a small number of layers (e.g., 2 or 3) to adjust by  $\pm 2$  bits (within the allowable range  $\{2, 4, 6, 8\}$ ). We then recalibrate and run a short QAT cycle.
- 3) **Re-Evaluate Metrics:** We check the global accuracy and model size usage. If both are within their buffers, we stop; if neither is acceptable, we keep adjusting.
- 4) **Early Stopping / Reversion:** If too many consecutive adjustments fail to move the metrics closer to their goals, we revert to the previous stable assignment and exit.

We use symmetric min–max range quantization for weights, performed per output channel (as implemented in Brevitas [41]), which is a common hardware-friendly scheme. For activations, we adopt asymmetric quantization with statistical clipping at the 99.9th percentile to reduce outlier sensitivity and improve calibration robustness. Unless stated otherwise, we follow a memory-centric objective (Model Size counted on weights

only), and thus keep activations fixed at 8 bits while allowing layer-wise weight bitwidths to vary within  $\{2, 4, 6, 8\}$  under SigmaQuant; when targeting compute (BOPs), both weights and activations can be adapted with the same procedure. Compared to recent MPQ methods that (i) co-design precision assignment with platform-specific inference units and pipelines (e.g., Edge-MPQ [32]), (ii) rely on second-order sensitivity (Hessian) and specialized transition-aware QAT (e.g., HMQAT [33]), (iii) explore a joint pruning–quantization space via reinforcement learning with an accelerator-level energy model [34], or (iv) formulate bitwidth assignment as a global integer quadratic program with cross-layer dependency modeling (e.g., CLADO [35]), SigmaQuant adopts a lightweight distribution-fitting strategy. More specifically, Phase 1 clusters layers using  $\sigma$  for a stable near-feasible initialization, and Phase 2 uses KL divergence to make small local bitwidth updates that explicitly enforce user-defined accuracy and resource constraints, without platform-specific co-design, Hessian estimation, RL exploration, or IQP solvers. By iterating this local adjustment approach, Phase 2 ensures that the model converges smoothly to a configuration that respects both accuracy and model size constraints without introducing large distributional mismatches at once; in practice, this step often accounts for relatively few bitwidth changes, given that Phase 1 has already established a near-feasible starting point.

## V. EXPERIMENTAL SETUP

To validate the effectiveness of *SigmaQuant* we conduct experiments on ImageNet [42] and CIFAR-100 datasets [43] using InceptionV3 [44] and five standard ResNet variants [45]—ResNet18, ResNet34, ResNet50, ResNet101, and ResNet152. We evaluate the proposed approach against state-of-the-art *heterogeneous* quantization approaches as well as conventional *uniform* quantization methods. To compare the result with other state-of-art method, we run experiments using the ImageNet (See Sec. VI-B). To show the trend and relation between the model size and accuracy, we run an experiment using CIFAR-100 to save GPU resources (see Sec. VI-C). We acquire the full-precision pre-trained models from the pytorch official website[46] for ImageNet. For CIFAR-100, we train these models again ourselves on the target dataset before quantization using either:

- **Uniform Quantization:** All layers share a fixed bitwidth (e.g., 2, 4 or 8 bits).
- **SigmaQuant (Ours):** Layers receive bitwidths guided by their weight distribution properties (standard deviation and optional divergence checks).

Before quantization, a short calibration phase (using a small subset of the training set) is performed. Then QAT is used to mitigate performance degradation.

The metrics used to assess the different techniques are the following:

- **Model Size (MB):** Sum of quantized weights across all layers.
- **Top-1 Accuracy (%):** Evaluated on the CIFAR-100 test set.

- **Regression Analysis:** For combined sets of quantized models, we fit accuracy–model-size curves and plot error bands (standard deviation) to visualize overall trends.

Moreover, to validate the hardware impact from *SigmaQuant*, we map the Resnet family on a general shift-add-based MAC unit and evaluate the latency and power consumption information through post-synthesis simulation with TSMC 28nm library. Our experiments were run on NVIDIA A100 and V100 GPUs, and the results were validated through more than 8\*500 GPU hours.

## VI. RESULTS

In this section, we first illustrate how the quantization process evolves over time in our two-phase algorithm, highlighting the transition from cluster-based assignment to iterative refinement. Then, we compare the performance of our method against state-of-the-art heterogeneous quantization schemes. Next, we analyze the trade-off between model size and accuracy in CIFAR-100, providing information on resource-accuracy balance. In the following, we discuss the hyperparameter effect. Finally, we demonstrate the potential hardware advantages.

### A. Phase-Based Learning Process

To illustrate the dynamic process of our method, we track the quantization states (“start” to “end”) of a representative model (e.g., ResNet34) as it progresses through the two-phase algorithm described in Section IV.

Fig. 3 plots the model accuracy (in %) against the corrected model sizes (in megabyte), depicting the iterative exploration of the *bit increase*, *bit decrease*, and *target* regions:

- **Start Point – Conventional Clustering:** Initially, the bitwidth assignment is based on k-means clustering of layers. This coarse clustering provides an initial bitwidth assignment that quickly improves accuracy while causing the model size to approach the target buffer (Fig. 3 left).
- **Phase I – Iterative Re-Clustering:** In Phase I, the model re-clusters individual layer bitwidths using an adaptive clustering method (Eq. 2). This technique assigns bitwidths to a higher or lower cluster, transitioning the model from the bit-increasing zone to the bit-decreasing zone and subsequently into Phase 2 (Fig. 3 middle).
- **Convergence and Abort Conditions:** Once one of the target metrics (accuracy or model size) meets the buffer threshold, Phase 2 commences. Here, we compute a sensitivity score for each layer that combines its weight standard deviation with a normalized Kullback–Leibler divergence measure. Layers exhibiting minimal divergence, indicating that further precision reduction would incur only minor accuracy loss, are selectively adjusted (Fig. 3 right).

We fix the number of clusters to 4, corresponding to weight bitwidths of  $\{2, 4, 6, 8\}$ -bit. These choices are illustrative, and other configurations could equally be adopted. Phase 1 runs up to 2 iterations (this number can be configured to a higher



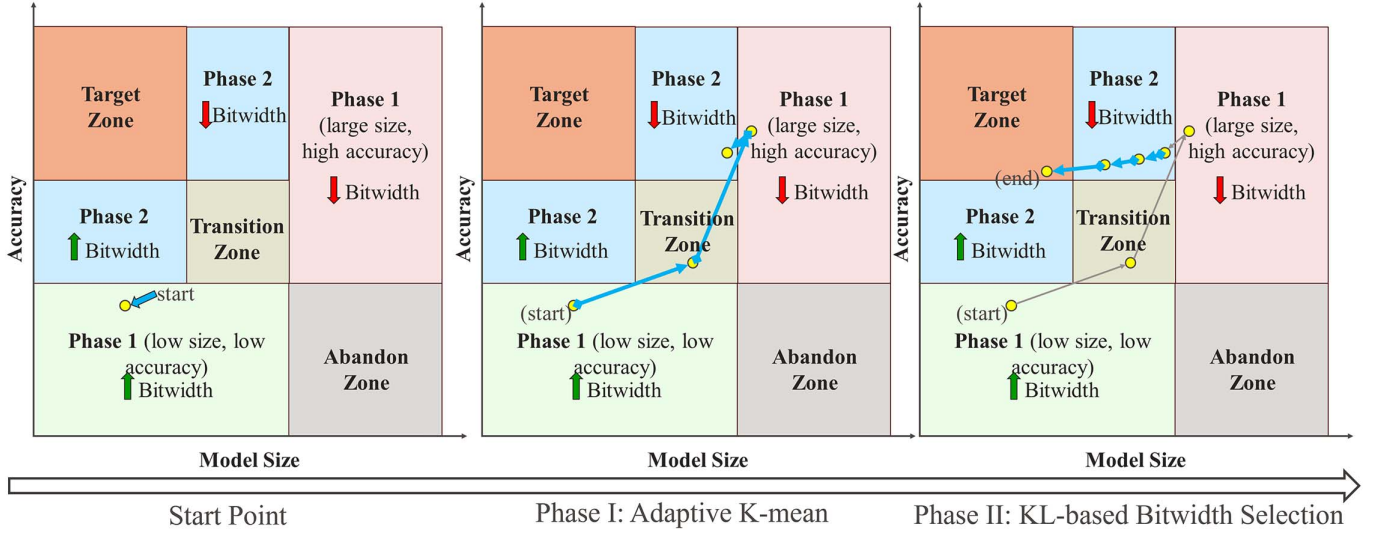


Fig. 3. Example showing how training advances through the two-phase quantization for ResNet34. The x-axis represents corrected model sizes and the y-axis represents model accuracy. Different points indicate successive stages in the cluster phase (phase 1) and iteration phase (phase 2), with the final quantized model landing in the target area.

value when facing larger models), each followed by 4 epochs of QAT. Phase 2 allows up to 40 refinement steps with 40 QAT epochs per step, adjusting 2 layers (we fixed this to 2 layers in our implementation for all experiments) per iteration based on sensitivity. QAT uses cross-entropy loss and SGD (for ResNet) or Adam (for others), with a reduced learning rate. We start from INT8 quantized models. The accuracy threshold is set to 1% drop, and the memory constraint targets 75% of the INT8 model size ( $\approx 18.75\%$  of the FP32 baseline). In general, Fig. 3 demonstrates the stability of the quantization path, showing that only a handful of iterations are typically required before a near-optimal trade-off between accuracy and model size overhead. Additionally, we compare the configuration obtained after Phase I alone with the final configuration after Phases I–II, to quantify the added benefit of Phase II. Table II summarizes results across models under a  $\leq 2\%$  accuracy-drop constraint and a  $\leq 40\%$  INT8-size budget. As noted, Phase 1—our fast, adaptive k-means clustering based solely on layer-wise standard deviation—can leave the model slightly above or below the memory target; the table therefore reports the Phase 1 (“std-only”) accuracy/size alongside the final result after Phase 2. After Phase 1 we choose a direction: either increase the bitwidth of the most sensitive layers to recover accuracy or decrease bitwidths where possible to meet the size budget. This operating region near 35–40% of the INT8 size is particularly challenging because further bitwidth reductions leave limited headroom to preserve accuracy. For ResNet-18, Phase 1 already satisfies the size budget but the accuracy drop is excessive; increasing the precision of selected layers ( $\uparrow$  in Table II) restores accuracy while keeping the model within budget, so both constraints are ultimately met. For ResNet-34, both constraints are met after Phase 1, so no refinement is required. For ResNet-50, the size target is met but the accuracy constraint remains violated even after Phase 2, indicating that the two targets cannot be satisfied simultaneously under this setting.

TABLE II  
MODEL SIZES AND ACCURACIES

| Model     | Int8 Size (MiB) | Int8 Acc. (%) | Final Acc. (%) | Final Size (MiB) | Phase I Acc. (%) | Phase I Size (MiB) | Next Phase   | Target Meet  |
|-----------|-----------------|---------------|----------------|------------------|------------------|--------------------|--------------|--------------|
| ResNet18  | 11.14           | 80.40         | 78.41          | 3.72             | 75.98            | 3.20               | $\uparrow$   | $\checkmark$ |
| ResNet34  | 20.77           | 82.90         | -              | -                | 81.14            | 6.52               | -            | $\checkmark$ |
| ResNet50  | 24.32           | 84.01         | 81.21          | 8.49             | 82.57            | 8.93               | $\downarrow$ | $\times$     |
| ResNet101 | 42.38           | 85.98         | 84.31          | 12.28            | 84.78            | 15.26              | $\downarrow$ | $\checkmark$ |
| ResNet152 | 57.26           | 86.77         | 84.67          | 16.72            | 85.26            | 22.66              | $\downarrow$ | $\times$     |

### B. Performance Comparisons With State-of-the-Art Heterogeneous Quantization Schemes

In this subsection, we compare our proposed quantization technique (**Ours**) against state-of-the-art methods on ResNet-50 [38] and InceptionV3 [44], focusing on model size (in MB) and Top-1 accuracy (%). We implement our method to achieve optimal performance under various memory constraints by setting target bitwidths to 2, 4, 6, and 8, resulting in a mixed-bitwidth configuration. Table III consolidates results from prior work [11], [12], [35], [36], [47], [48], [49] alongside our heterogeneous bitwidth assignments for weights (W) and activations (A) (column Bits(W,A) in the table). We emphasize weight precision while treating activations as 8-bit, since our primary objective is memory reduction. For each model, we list the baseline full-precision size and accuracy, then compare uniform or mixed-precision baselines to illustrate how different bit configurations trade off memory and accuracy. The methods labeled Ours highlight two representative configurations that demonstrate a favorable balance between model compactness and classification performance.

Focusing primarily on memory efficiency, these results show that our heterogeneous quantization yields models that are up to **7–10 $\times$  smaller** than the full-precision baseline, while retaining

TABLE III  
COMPARISON OF QUANTIZATION METHODS ON INCEPTIONV3  
AND RESNET-50

| Method             | Bits(W,A) | Model Size(MB) | Top-1 Acc.(%) |
|--------------------|-----------|----------------|---------------|
| <b>ResNet-50</b>   |           |                |               |
| Baseline [11]      | 32,32     | 97.8           | 77.72         |
| Apprentice [36]    | 2+, 8+    | 14.11          | 72.8          |
| UNIQ [12]          | 4,8       | 12.8           | 74.37         |
| UNIQ [12]          | 4+, 8+    | 12.8           | 75.1          |
| Apprentice [36]    | 4+, 8+    | 20             | 74.7          |
| Apprentice [36]    | 2+,32     | 14.1           | 74.7          |
| UNIQ [12]          | 4,32      | 12.8           | 75.09         |
| HAWQ-V3 [11]       | 8,8       | 24.5           | 77.58         |
| HAWQ-V3 [11]       | 4/8,4/8   | 18.7           | 76.73         |
| HAWQ-V3 [11]       | 4,4       | 13.1           | 74.24         |
| CLADO [35]         | mix, 8    | 17.89          | 75.42         |
| CLADO [35]         | mix, 8    | 13.42          | 73.10         |
| Ours               | mix,8     | 12.02          | 76.86         |
| Ours               | mix,8     | 10.78          | 75.63         |
| <b>InceptionV3</b> |           |                |               |
| Baseline [11]      | 32,32     | 90.9           | 78.88         |
| Integer Only [8]   | 8, 8      | 22.7           | 74.20         |
| Integer Only [8]   | 7, 7      | 20.1           | 73.70         |
| RVQuant [12]       | 8,8       | 22.7           | 74.22         |
| HAWQ-V3 [11]       | 4/8,4/8   | 19.6           | 74.65         |
| Ours               | mix,8     | 19.63          | 74.73         |

most of the original accuracy. For example, for ResNet-50, *Ours* (mix,8) (12.02 MB) exceeds 76% accuracy, aligning with methods that use similar or slightly larger model sizes, while 13.1MB for HAWQ-V3 with 74.24% accuracy, 13.42MB for CLADO with 73.10% accuracy. Similarly, for InceptionV3, our method reach similar Model Size (19.6MB for HAWQ-V3, 19.63MB for Ours) and Top-1 accuracy (74.65% for HAWQ-V3, 74.73% for Ours) as the HAWQ-V3. Notably, we achieve these gains using a straightforward procedure that incorporates standard deviation and divergence metrics to adapt the bitwidth per layer. Compare to the HAWQ-V3, our method support more bitwidths choices, and it can shrink the model size adaptively according to the memory constrain (as shown in Fig. 4(b)). These comparisons confirm that our method effectively balances memory footprint with competitive accuracy, making it a suitable choice for scenarios where model size is the foremost constraint. In our design, SigmaQuant’s search cost is dominated by short QAT loops rather than by an expensive discrete search. Phase 1 computes per-layer statistics (one scalar  $\sigma$  per layer) and runs adaptive  $k$ -means on these  $L$  layers with  $K = 4$  clusters, which is limited in practice ( $M$  iterations for example). Computing KL divergences uses layer histograms and scales with the number of parameters  $P$  once per refinement round. Phase 2 makes small local moves—changing  $m$  layers per round ( $m = 2$  in our setup)—and applies brief QAT to re-stabilize accuracy. Overall, the wall-clock is therefore well approximated by the number of QAT epochs:

$$\text{Cost} \approx (M E_{P_1} + N E_{P_2}) \times T_{\text{epoch}},$$

where  $M$  is the number of phase 1 rounds,  $E_{P_1}$  are Phase 1 epochs,  $N$  is the number of refinement rounds,  $E_{P_2}$  are epochs per round, and  $T_{\text{epoch}}$  is the time for one QAT epoch. In our

experiments we cap Phase 1 at normally 1 to 3 iterations and Phase 2 runs normally within 5 (smaller models) to 40 (large models) refinement rounds, with early stopping once both accuracy and size targets are met. As a result, end-to-end times are model-dependent: for the runs in this paper we measured  $\approx 2$  to 30 hours for ResNet-18/34/50/101/152, respectively, using A100/V100 GPUs (see Sect. VI-C). We do not claim to be faster than calibration-only PTQ methods that avoid any fine-tuning; instead, SigmaQuant trades a moderate offline search for two properties that those methods typically lack: (i) hard-constraint adaptivity (meeting user-specified accuracy/size targets across devices) and (ii) consistently better accuracy–size and hardware PPA trade-offs (Tables II, III, Fig. 4). Methods such as HAWQ-V3 estimate per-layer sensitivity via second-order information and solve a global assignment with ILP; their runtime is dominated by many backward passes for Hessian/spectrum estimation plus calibration, after which assignment is relatively cheap. UNIQ and Apprentice require training (noise injection or distillation) across multiple epochs; they do not perform a per-layer mixed-precision search but still incur substantial wall-clock due to full-model optimization.

SigmaQuant lies in between: it avoids Hessian estimation and global RL/ILP search, yet it still adapts layer precisions to explicit hardware/accuracy constraints via a small number of targeted QAT loops. In short, SigmaQuant is not a “zero-search” PTQ method; it is an adaptive mixed-precision method with linear-in-layers bookkeeping and a bounded number of short QAT epochs, which empirically yields favorable accuracy–size–hardware trade-offs under tight device constraints.

### C. Model Size and Accuracy Analysis

Another key advantage of our method is its adaptive nature. By dynamically adjusting the boundary conditions for both the model size buffer and the accuracy target, our approach performs layer-wise quantization in a highly flexible manner, a capability that previous methods lack. Fig. 4(a) compares model size and Top-1 accuracy on CIFAR-100 for five different ResNet architectures (ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152) under two quantization schemes: conventional uniform quantization and our *SigmaQuant* approach. Across all model variants, *SigmaQuant* (plotted as darker markers) achieves higher accuracy for a given model size than their uniformly quantized counterparts (lighter markers). In practical terms, for an equal memory footprint, a model quantized with *SigmaQuant* yields 4% more accuracy. For the same accuracy, the model size can shrink by around 40%. Notably, the uniform quantization approach is unable to reach the optimal balance between model size and accuracy, highlighting its inherent limitations. This trend holds consistent as networks become deeper: model quantized with our method surpasses the accuracy of a similarly-sized uniformly quantized one. For example, the ResNet50 sigma can achieve similar accuracy (81.0% for uniform quantization, 81.7% for sigma quantization) with better model size (18.24MB for uniform quantization, 13.98MB for sigma quantization). In effect, *SigmaQuant* retains

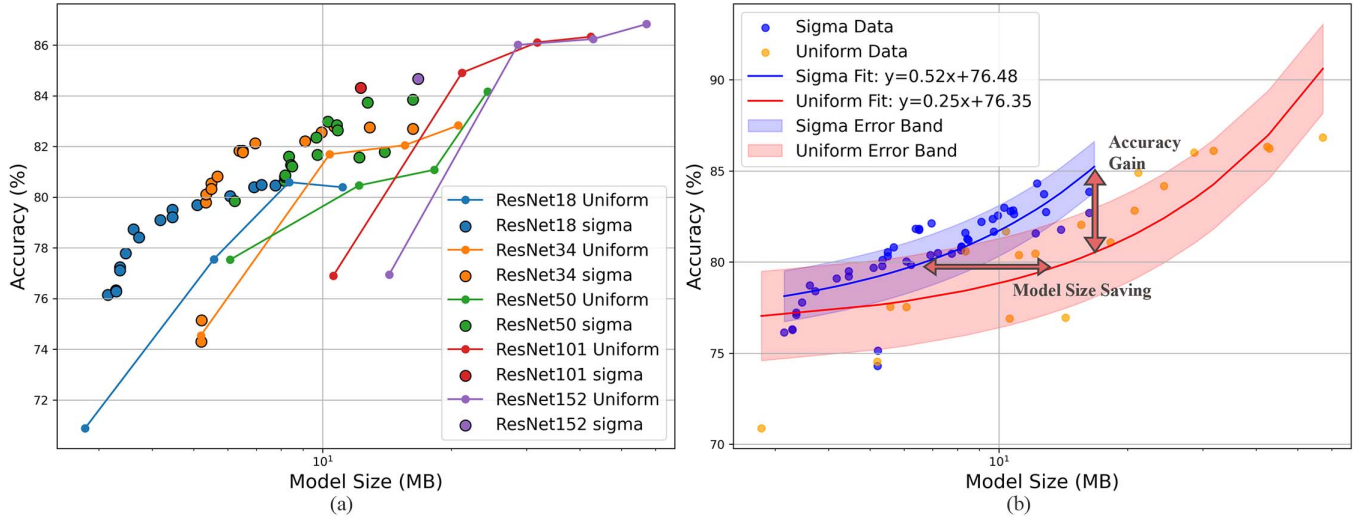


Fig. 4. (a) Comparison of top-1 accuracy versus model size for various ResNet architectures on CIFAR-100, where darker markers denote the *sigma*-based method and lighter markers denote uniform quantization. (b) Regression fits with  $\pm 1\sigma$  error bands reveal that the *sigma* approach consistently achieves higher accuracy at equivalent model sizes.

and, in some cases even improves, the predictive performance while aggressively reducing the size of the model.

To further illustrate the overall trend, Fig. 4(b) consolidates the results by plotting top-1 accuracy against model size for all quantized ResNet models tested, with linear regression fits for each quantization scheme (*sigma*-based vs. uniform) and shaded  $\pm 1\sigma$  error bands. The key observation is that the fitted curve for *SigmaQuant* lies consistently above that of uniform quantization across the entire range. The separation between these trendlines is substantial, and the error bands show minimal overlap, indicating that the accuracy gap is statistically robust across multiple runs. In other words, *SigmaQuant* achieves a given accuracy level with fewer parameters (3.2MB *Model Size Saving* in Fig. 4(b)), or conversely, for the same number of parameters (same memory budget), it delivers higher accuracy than uniform precision (4% *Accuracy Gain* in Fig. 4(b)). This translates to a more favorable accuracy–efficiency trade-off. Notably, the advantage becomes even more pronounced for larger models: *SigmaQuant*’s accuracy approaches that of higher-precision (e.g., 32-bit) networks at a fraction of the memory usage as model size grows. Such behavior is highly desirable for resource-constrained deployments, as it means that one can enjoy near-baseline accuracy without the cost of a large model. Overall, the regression fits and non-overlapping error bands indicate that *SigmaQuant* consistently provides a better accuracy–model-size trade-off than uniform quantization across different ResNet depths.

Our experiments also demonstrate that *SigmaQuant* remains practical for offline training across the ResNet family. For example, under an extreme constraint, limiting the model size to achieve at least 25% memory savings while tolerating at most 1% accuracy drop relative to the full precision model, the total quantization search and training times were approximately 4, 2, 1.5, 5.5, and 26 hours for ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152, respectively. These runtimes

remain within a reasonable range for offline deployment. Even in cases where the training process fails due to overly tight memory constraints, we observe from the generated models that *SigmaQuant* still produces meaningful accuracy–efficiency trade-offs, as shown in the corresponding evaluation figures.

#### D. Hyperparameter and the Activation

We study the impact of the accuracy/size buffers ( $\Delta_A$ ,  $\Delta_M$ ) and the refinement schedule (max rounds  $N_{\max}$ , step size  $m$  layers/round, and QAT epochs/round  $E_{P2}$ ) on convergence and wall-clock. Recall that Phase 1 ends once *either* of the metrics enters its buffer (Alg. 1, lines 12–16), whereas Phase 2 continues until both strict targets are met (lines 27–31). Thus, larger buffers generally reduce Phase 1 iterations but may require more Phase 2 refinements; smaller buffers have the opposite effect. We keep  $K=4$  clusters and the default 2-layer step in Phase 2 unless otherwise stated. Table IV summarizes typical behavior on **ResNet-34/CIFAR-100** under our default targets (at most 1% accuracy drop), varying only ( $\Delta_M$ ) and refinement settings. The trends hold qualitatively for other models. Overall, smaller buffers (*Conservative*) reduce the chance of overshooting a target but increase observed rounds  $N$  and wall-clock; larger buffers (*Aggressive*) do the opposite, at a slight risk of needing extra micro-adjustments later in Phase 2. In all cases, the final solution respects the strict targets due to the Phase 2 stopping rule.

Moreover, except the weights for the memory saving, we adapt the activation to reduce the amount of BOPs. To evaluate activation reduction fairly, we switch the target from memory to compute:

$$\text{BOPs} \triangleq \sum_{\ell} B_w(\ell) B_a(\ell) \text{MACs}(\ell),$$

where the  $B_w(\ell)$  and  $B_a(\ell)$  are the bitwidth of weight and activation; MACs is the multiply–accumulate operation of this



TABLE IV  
SENSITIVITY OF SIGMAQUANT ON RESNET-34 (CIFAR-100) UNDER THE  
DEFAULT TARGETS. NUMBERS SHOWN ARE PLACEHOLDERS

| Setting            | $\Delta_A$ | $\Delta_M$ | Obs. $M$ | Obs. $N$ | Time (h)    | Meet? |
|--------------------|------------|------------|----------|----------|-------------|-------|
| Conservative       | 1%         | 85%        | 3        | 0        | $\sim 4.5$  | ✓     |
| Balanced (default) | 1%         | 75%        | 3        | 3        | $\sim 12.6$ | ✓     |
| Aggressive         | 1%         | 50%        | 4        | 5        | $\sim 19.0$ | ✗     |

TABLE V  
ACTIVATION REDUCTION UNDER  
A BOP TARGET

| Model     | $\Delta_A$ | $\Delta_{BOP}$ |
|-----------|------------|----------------|
| ResNet-18 | 78.91%     | (−32.9%)       |
| ResNet-34 | 81.31%     | (−49.4%)       |
| ResNet-50 | 82.38%     | (−32.0%)       |

layer. It upper-bounds bit-level work and correlates with energy/latency (on our shift-and-add MACs, cycles scale primarily with  $B_w$ ) while exposing the benefit of lowering  $B_a$ . We then run SigmaQuant with a compute budget (BOPs) and an accuracy target, letting both weights and activations adapt. Table V reports indicative results on **AlexNet/ResNet-18/ResNet-32/ResNet-50** on CIFAR-100 with a 1% accuracy-drop target and a 25–35% BOPs-reduction budget. As expected, activation down-quantization reduces BOPs but leaves Model Size (MB) unchanged; SigmaQuant co-adapts layerwise weights to maintain accuracy. When the optimization target is memory (weight size), changing activation bitwidth has no effect on Model Size (MB) because activations are not counted in that metric. Under a compute target (BOPs), SigmaQuant naturally co-optimizes toward lower activation precision while preserving accuracy, yielding 35–50% BOPs reductions in our indicative runs with  $\leq 2.5\%$  accuracy loss.

Overall, these results show that our distribution-guided *SigmaQuant* method provides an effective balance between model compression and predictive performance. By judiciously allocating bitwidths based on per-layer weight statistics and distributional divergence, *SigmaQuant* consistently produces more compact models without sacrificing accuracy under the same model-size constraints. This represents a compelling improvement over uniform quantization, making our approach well-suited for real-world DNN deployment in memory-limited embedded environments.

### E. Hardware Performance Analysis

A key advantage of heterogeneous quantization is that it leverages the varying robustness of different layers to maintain accuracy while allowing the corresponding hardware to operate with less area occupation, fewer cycle count, and lower energy consumption, increasing all benefits.

In this work, we target a widely used shift-add MAC implementation to evaluate the performance under different quantization schemes. This MAC unit has an 8-bit shift-add-based multiplier that supports  $8\text{-bit} \times n\text{-bit} \rightarrow 8\text{-bit}$  multiplication through

TABLE VI  
MAC IMPLEMENTATIONS

|                        | FP32               | FP16                | BF16                | INT8                 | Shift-Add                   |
|------------------------|--------------------|---------------------|---------------------|----------------------|-----------------------------|
|                        | 32-bit             | 32-bit              | 32-bit              | 32-bit               | 32-bit                      |
| Multiplication         | 1 subword<br>FP32× | 2 subwords<br>FP16× | 2 subwords<br>BF16× | 4 subwords<br>INT8×  | 4 subwords<br>8-bit $\gg$ + |
| Accumulation           | 1 subword<br>FP32+ | 2 subwords<br>FP32+ | 2 subwords<br>FP32+ | 4 subwords<br>INT32+ | 4 subwords<br>INT32+        |
| Area / $\mu\text{m}^2$ | 3218.3             | 3837.9              | 3501.9              | 2103.4               | 1635.4                      |

iterative right-shift and addition, followed by a 32-bit adder for accumulation. The shift-add design performs a single addition and multiple right-shift operations within each cycle, which enables to process trailing zeros of the multiplier operand, reducing the average latency to roughly  $n/2$  cycles for an  $n$ -bit operand. The shift-add MAC hardware is implemented in TSMC 28nm technology (0.9V, 600MHz) and features a 32-bit datapath. It should be emphasized that the arithmetic performance comparison reflect a general case applicable to all technology nodes and hardware platforms, while the presented implementation is intended primarily for illustrative purposes. We also implemented FP32, FP16, BF16, and INT8 alternatives under the same conditions, detailed in Table VI. It can be found that shift-add MAC implementation reduces 22.3% area over the INT8 one, and more than 49.2% over others. We characterize the performance using post-synthesis simulation, mapping all convolutional and fully-connected layers of ResNet models onto the hardware. The energy consumption and cycle count for inference are evaluated, as shown in Fig. 5. In Fig. 5, we show the normalized energy consumption (top, a) and cycle count (hence latency, (bottom, b)) of different quantization schemes across benchmarks. Both our *SigmaQuant* scheme and the uniform quantization approach are mapped on the shift-add design, whereas the typical INT8 quantization is performed with the INT8 hardware that already has high efficiency. The results of FP32, FP16, BF16 are not included, since they incur up to  $5.5\times$ ,  $4.0\times$ , and  $3.6\times$  more energy cost over the INT8 one, respectively. The uniform quantization approach has four combinations of 8-bit activations and 2/4/6/8-bit weights, namely as A8W8, A8W6, A8W4 and A8W2. All the values are normalized by the INT8 MAC implementation. The accuracy drop is defined as the difference between FP32 reference and the quantized ones.

As expected, in uniform quantization, when weights reduce from 8-bit to 4-bit, and especially to 2-bit, the accuracy dramatically drops. Taking Resnet34 as an example, even if uniform quantization A8W2 saves 25.0% energy consumption wrt. the INT8 case, it degrades accuracy by 8.54%. However, *SigmaQuant* performs a larger search in the design space and is able to address this issue, achieving 23.3% energy savings over the INT8 alternative with only a 2.97% accuracy loss. As for the A8W4 case, uniform quantization can save 13.8% energy consumption with 1.39% accuracy loss while our method reduces 16.0% energy cost with only 1.25% accuracy degradation. Furthermore, the results of *SigmaQuant* are consistent across all the benchmarks, always yielding data points located closer to the

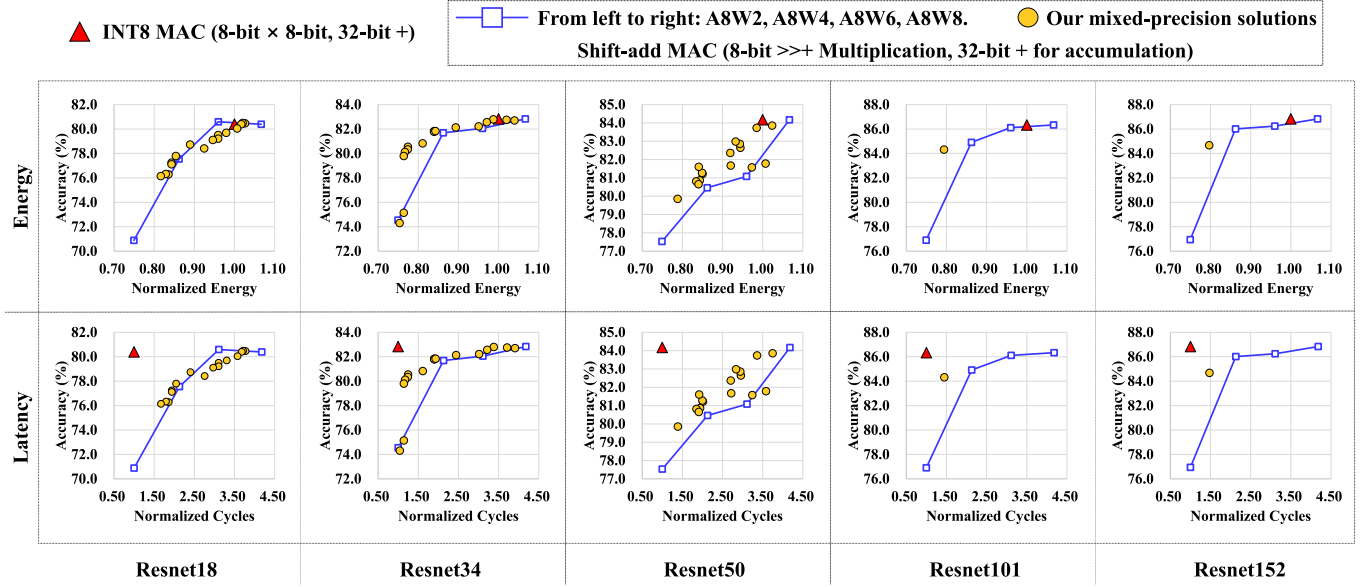


Fig. 5. Normalized energy consumption (top) and cycle count (hence latency, bottom) versus accuracy for all ResNet models with uniform quantization (A8W2, A8W4, A8W6, A8W8) and sigma quantization schemes. An INT8 MAC, which has 1-cycle  $8\text{-bit} \times 8\text{-bit}$  multiplier and 32-bit adder, is used as a baseline for performance comparison. FP32, FP16, and BF16 alternatives are not presented here since they incur huge overhead, that up to  $5.5\times$ ,  $4.0\times$ , and  $3.6\times$  more energy cost. All the data are normalized with INT8 computation. The closer a dot element is to the top-left corner, the better its performance and accuracy.

upper left corner in the figure wrt. the uniform approach, hence less latency and energy consumption with lower accuracy loss. The only exception is ResNet18, where *SigmaQuant* shows a very similar trend to uniform quantization, since such model is too small and therefore the space search is limited. Still, *SigmaQuant* can get 15.7% less energy consumption wrt. the INT8 case with 3.76% accuracy drop while that of uniform quantized one is 13.8% and 3.45%. For larger models such as ResNet101 and ResNet152, that provides enough searching space, our *SigmaQuant* method can reduce up to 20.6% and 20.3% energy cost wrt. the INT8 alternative, which is already a well-known solution, with comparable accuracy. Note that the model size of our method is always smaller than that of INT8 quantization for all the comparisons.

Similar trend can be observed in the latency performance. Due to the serial shift-add-based multiplication, even multiple shifts per cycle is used, it still incurs latency overhead. For example, the A8W8 uniform quantization of ResNet34, runs  $4.2\times$  slower than the INT8 case. Thanks to the fully exploitation and optimization by *SigmaQuant*, however, it dramatically cuts this overhead. For ResNet34, it lowers energy by 23.3% while increasing latency by only 17.5% relative to the INT8 alternative. This analysis only considers the general shift-add arithmetic, yet the advantages would become far more pronounced with specific techniques such as CSD encoding [16] applied to the multiplier operand.

In addition, uniform quantization offers a very limited range of choices, making it difficult to balance accuracy with latency and energy efficiency. In contrast, the *SigmaQuant* dot distribution shown in Fig. 5 illustrates a broader set of quantized models. For example, the options between A8W2 to A8W4 or

A8W4 to A8W6 for ResNet18, ResNet34, and ResNet50. This greater flexibility allows for more effective tailoring to diverse hardware requirements, including model size, runtime latency, and energy budget.

Overall, these results demonstrate that distribution-guided, layer-wise bit allocation can significantly enhance hardware efficiency compared to uniform quantization. In fact, our approach shows greater improvements than the A8W4 variant, approaching the efficiency of A8W2 quantization while incurring minimal accuracy loss, and can further improve hardware efficiency than typical INT8 solution. As model size decreases, fine-grained quantization leverages the shift-add architecture to achieve lower latency and reduced power consumption, offering a compelling solution for resource-constrained deployments.

## VII. CONCLUSION

In this work, we have presented *SigmaQuant*, an adaptive, layerwise heterogeneous quantization framework that leverages a distribution-fitting approach to assign bitwidths based on both weight standard deviation and KL divergence. Our two-phase method, featuring an initial cluster-based bitwidth assignment followed by an iterative, divergence-driven refinement, effectively balances accuracy and model size with a reasonable searching effort, ensuring that quantized models maintain high performance while achieving significant compression. A key strength of *SigmaQuant* is its inherent adaptability. Across different platforms with varying constraints and requirements, our framework consistently adapts bit allocation to satisfy specific resource budgets. For example, by integrating

with hardware-centric metrics such as model size and target accuracy, *SigmaQuant* directly translates per-layer quantization decisions into efficient bitwidth assignments that align with the overall requirements of the system. In general, our experimental results on CIFAR-100 and ImageNet across multiple DNN architectures demonstrate that *SigmaQuant* not only outperforms state-of-the-art heterogeneous quantization and uniform quantization, but also provides a robust and hardware-aware solution for efficient DNN inference in resource-limited environments. Furthermore, experiments on general hardware arithmetic scheme validate our results from both power and latency perspectives, further emphasizing the advantages of our approach. This adaptive, dynamic method paves the way for the practical deployment of deep neural networks in a wide range of embedded systems.

## REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.
- [2] D. Amodei et al., "Deep speech 2: end-to-end speech recognition in English and mandarin," in *Proc. 33rd Int. Conf. Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 173–182.
- [3] Q. Liu, M. Zapater, and D. Atienza, "MatrixFlow: System-accelerator co-design for high-performance transformer applications" 2025, *arXiv:2503.05290*. [Online]. Available: <https://arxiv.org/abs/2503.05290>
- [4] Q. Liu, M. Zapater, and D. Atienza, "Gem5-AcceSys: Enabling system-level exploration of standard interconnects for novel accelerators," 2025, *arXiv:2502.12273*. [Online]. Available: <https://arxiv.org/abs/2502.12273>
- [5] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
- [6] L. Deng, "Model compression and acceleration for deep neural networks: The principles, progress, and challenges," *IEEE Signal Process. Mag.*, vol. 37, no. 6, pp. 101–110, 2020.
- [7] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2016, pp. 1–14.
- [8] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [9] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," 2018, *arXiv:1806.08342*.
- [10] R. Banner, I. Hubara, E. Hoffer, and D. Soudry, "Scalable methods for 8-bit training of neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018, pp. 5145–5153.
- [11] Z. Yao et al., "HAWQ-V3: Dyadic neural network quantization," in *Proc. 38th Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 11875–11886.
- [12] C. Baskin et al., "UNIQU: Uniform noise injection for non-uniform quantization of neural networks," *ACM Trans. Comput. Syst.*, vol. 37, nos. 1–4, pp. 1–15, 2021.
- [13] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Piscataway, NJ, USA: IEEE Press, 2016, pp. 1–12.
- [14] L.-C. Hsu, C.-T. Chiu, K.-T. Lin, H.-H. Chou, and Y.-Y. Pu, "ESSA: An energy-aware bit-serial streaming deep convolutional neural network accelerator," *J. Syst. Archit.*, vol. 111, 2020, Art. no. 101831.
- [15] M. Rios, F. Ponzina, A. Levisse, G. Ansaloni, and D. Atienza, "Bit-line computing for CNN accelerators co-design in edge ai inference," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 2, pp. 358–372, 2023.
- [16] P. Yu et al., "An energy efficient soft SIMD microarchitecture and its application on quantized CNNs," *IEEE Trans. Very Large Scale Integr.*, vol. 32, no. 6, pp. 1018–1031, Jun. 2024.
- [17] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "HAWQ: Hessian aware quantization of neural networks with mixed precision," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 293–302.
- [18] A. Yazdanbakhsh, A. T. Elthakeb, P. Pilligundla, F. S. Miresghallah, and H. Esmailzadeh, "ReLeQ: A reinforcement learning approach for deep quantization of neural networks," 2019, *arXiv:1811.01704*.
- [19] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," University of Toronto, Toronto, Ontario, Canada, Tech. Rep., 2009.
- [20] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
- [21] E. Park, J. Ahn, and S. Yoo, "Weighted-entropy-based quantization for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 7197–7205.
- [22] X. Zhu, Q. Li, and D. Wang, "Entropy-based layerwise quantization for efficient deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 1234–1243.
- [23] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 8604–8612.
- [24] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 1, pp. 1997–2017, Jan. 2019.
- [25] B. Wu et al., "FBNet: Hardware-aware efficient convnet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 734, 2018, pp. 10–726. [Online]. Available: <https://api.semanticscholar.org/CorpusID:54461508>
- [26] H. Cai, M. F. Kaloorazi, and J. Chen, "Online generalized eigenvectors extraction via a fixed-point approach," *IEEE Trans. Signal Process.*, vol. 69, pp. 2435–2451, 2021.
- [27] Z. Dong, Z. Yao, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer, "Hawq-v2: Hessian aware trace-weighted quantization of neural networks," in *Proc. Adv. Neural Information Process. Syst.*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 18518–18529. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/2020/file/d77c703536718b95308130ff2e5cf9ee-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2020/file/d77c703536718b95308130ff2e5cf9ee-Paper.pdf)
- [28] L. Wang and B. Chen, "Adabits: Adaptive bitwidth quantization for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 5560–5568.
- [29] A. Finkelstein, U. Almog, and M. Grobman, "Fighting quantization bias with bias," 2019, *arXiv:1906.03193*. [Online]. Available: <https://api.semanticscholar.org/CorpusID:174801144>
- [30] C. Hong and K. M. Lee, "Overcoming distribution mismatch in quantizing image super-resolution networks," in OpenReview, 2024. [Online]. Available: <https://openreview.net/forum?id=GOt2kp383R>
- [31] A. Bulat and G. Tzimiropoulos, "Bit-Mixer: Mixed-precision networks with runtime bit-width selection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Los Alamitos, CA, USA: IEEE Computer Soc., Oct. 2021, pp. 5168–5177. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICCV48922.2021.00514>
- [32] X. Zhao, R. Xu, Y. Gao, V. Verma, M. R. Stan, and X. Guo, "Edge-MPQ: Layer-wise mixed-precision quantization with tightly integrated versatile inference units for edge computing," *IEEE Trans. Comput.*, vol. 73, no. 11, pp. 2504–2519, Nov. 2024.
- [33] Z. Huang, X. Han, Z. Yu, Y. Zhao, M. Hou, and S. Hu, "Hessian-based mixed-precision quantization with transition aware training for neural networks," *Neural Netw.*, vol. 182, 2025, Art. no. 106910.
- [34] K. Balaskas et al., "Hardware-aware DNN compression via diverse pruning and mixed-precision quantization," *IEEE Trans. Emerg. Topics Comput.*, vol. 12, no. 4, pp. 1079–1092, Oct./Dec. 2024.
- [35] Z. Deng, S. Sharify, X. Wang, and M. Orshansky, "Mixed-precision quantization for deep vision models with integer quadratic programming," in *Proc. 62nd ACM/IEEE Des. Autom. Conf. (DAC)*, 2025, pp. 1–7.
- [36] A. Mishra and D. Marr, "Apprentice: Using knowledge distillation techniques to improve low-precision network accuracy," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2018, pp. 1–17.
- [37] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [39] M. Federici, R. Tomioka, and P. Forré, "An information-theoretic approach to distribution shifts," in *Proc. 35th Int. Conf. Neural Information Process. Syst. (NIPS)*, Red Hook, NY, USA: Curran Associates Inc., 2021, pp. 1–14.



- [40] M. Rios, W. Simon, A. Levisse, M. Zapater, and D. Atienza, "An associativity-agnostic in-cache computing architecture optimized for multiplication," in *Proc. IFIP/IEEE 27th Int. Conf. Very Large Scale Integration (VLSI-SoC)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 34–39.
- [41] G. Franco, A. Pappalardo, and N. J. Fraser, "Xilinx/brevitas," 2025, doi: 10.5281/zenodo.3333552.
- [42] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [43] A. Krizhevsky and G. Hinton, "Cifar-100 dataset," 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [44] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Los Alamitos, CA, USA: IEEE Computer Soc., Jun. 2016, pp. 2818–2826, doi: 10.1109/CVPR.2016.308.
- [45] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [46] PyTorch, "Models — torchvision documentation," [Online]. Available: <https://pytorch.org/vision/stable/models.html>
- [47] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen, "Incremental network quantization: Towards lossless CNNs with low-precision weights," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.
- [48] Y. Xu, Y. Wang, A. Zhou, W. Lin, and H. Xiong, "Deep neural network compression with single and multiple level quantization," in *Proc. AAAI Conf. on Artif. Intell. (AAAI-18)*, 2018, pp. 4335–4342, 2018.
- [49] A. Polino, R. Pascanu, and D. Alistarh, "Model compression via distillation and quantization," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2018, pp. 1–21.